

CSE5095: Types and Programming Languages

Homework 2: Type Safety

Prof. Stefan Muller

Out: Thursday, Sep. 10

Due: Thursday, Sep. 24 11:59pm EDT

This assignment contains 10 tasks, for a total of 60 points, plus a bonus task worth up to 3 points.

1 Logistics

Please read and follow these instructions carefully.

Writing up Solutions. Please submit your homework writeup as a .pdf or .doc document (.pdf is preferred). Typesetting your answers using LaTeX is strongly recommended, as it produces the easiest-to-read documents. Typed answers using Word or other tools, or neatly handwritten and scanned answers, are acceptable. (Note that Word, Google Docs, etc., make it unreasonably difficult to write inference rules, which you'll be doing a lot).

Collaboration and Academic Honesty. This homework is to be completed individually. See the syllabus for more detail on the collaboration policy, and in particular the policy on AI.

2 Booleans

In this task, you'll extend the E language from class with Booleans:

$$\begin{array}{ll} \text{Types } \tau & ::= \text{int} \mid \text{string} \mid \text{bool} \\ \text{Expressions } e & ::= \bar{n} \mid \text{"s"} \mid \text{true} \mid \text{false} \mid e = e \mid e + e \mid e \wedge e \mid |e| \mid \text{if } e \text{ then } e \text{ else } e \end{array}$$

The expressions `true` and `false` are values and have their usual meanings. The expression `if  $e_1$  then  $e_2$  else  $e_3$` should evaluate e_1 . If it evaluates to `true`, it should continue evaluating e_2 , otherwise it should continue evaluating e_3 . The other branch *should not* be evaluated (this isn't possible to express in E^1 , but consider the code `if  $x = \bar{0}$  then  $\bar{0}$  else  $\overline{42}/x$` : we certainly don't want to evaluate the else branch when the condition is true!). We've also added an integer equality test $e_1 = e_2$ to have an interesting way of producing Booleans (this is an operation on integers only, not on Booleans or strings). Below are the additional rules for equality tests. Assume we still have all of the step rules from class, unchanged.

$$\begin{array}{ccc} \frac{}{\bar{n} = \bar{n} \mapsto \text{true}} \text{ (STPEQ)} & \frac{n_1 \neq n_2}{\bar{n}_1 = \bar{n}_2 \mapsto \text{false}} \text{ (STEPNOTEQ)} & \frac{e_1 \mapsto e'_1}{e_1 = e_2 \mapsto e'_1 = e_2} \text{ (STEPSEARCHEQLEFT)} \\ & & \frac{e_2 \mapsto e'_2}{\bar{n}_1 = e_2 \mapsto \bar{n}_1 = e'_2} \text{ (STEPSEARCHEQRIGHT)} \end{array}$$

¹Until later in this homework...

Task 1 (10 points).

Write the inference rules for the dynamic semantics of Booleans and the if-else construct. You should have 2 new rules for the judgment $e \text{ val}$ and 3 new rules for the judgment $e \mapsto e$.

(**Hint:** only one of these will be a search rule.)

Task 2 (3 points).

Evaluate the following expression fully (i.e., until you reach a value) using the dynamics semantics from class plus the ones you defined in the previous task, showing each step:

if $\bar{1} + \bar{2} = \bar{4}$ then “Fake news” else “True fact”

(That’s the end of Task 2; text for other tasks continues below.)

We also extend the typing rules with the new rule for equality testing and Booleans.

$$\frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 = e_2 : \text{bool}} \text{ (TYPEEQ)} \quad \frac{}{\text{true} : \text{bool}} \text{ (TYPETRUE)} \quad \frac{}{\text{false} : \text{bool}} \text{ (TYPEFALSE)}$$

$$\frac{e_1 : \text{bool} \quad e_2 : \tau \quad e_3 : \tau}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{ (TYPEIF)}$$

Note that rule TYPEIF doesn’t require that the two branches of the conditional have a particular type (e.g. `int`). The use of the same metavariable τ *does* however, mean that the two branches e_2 and e_3 must have the *same* type, which is then the type of the whole expression (this makes sense: how could we possibly give a type to the expression `if  $n = \bar{0}$  then  $\bar{4}\bar{2}$  else “Oops”?`).

Task 3 (5 points).

Give a derivation using the extended typing rules for the following statement (which gives a type to the same expression from above):

if $\bar{1} + \bar{2} = \bar{4}$ then “Fake news” else “True fact” : `string`

(You can abbreviate rule names, e.g. `TI` instead of `TypeIf`, to make things fit; if that still doesn’t work, you can use macros like `\scriptsize` to make things smaller)

Task 4 (7 points).

Prove the cases of the Preservation theorem for the new rules you added in Task 1.

Task 5 (3 points).

State (you do not have to prove it) the new case of the Canonical Forms lemma for Booleans.

Task 6 (8 points).

Prove the cases of the Progress theorem for the new rules TYPETRUE, TYPEFALSE and TYPEIF. You may (and should) use the new case of Canonical Forms from Task 5.

3 Division by zero

In class and the previous set of tasks, we used type systems and the ideas of type safety (progress and preservation) to prevent run-time type errors. Here, we’ll use a type system to prevent another run-time error: division by zero. Consider a language we’ll call `EZ`, whose syntax and dynamics are below.

$$\begin{aligned}\tau &::= \text{nz} \mid \text{mz} \\ e &::= \bar{n} \mid e + e \mid e - e \mid e * e \mid e / e\end{aligned}$$

$$\begin{aligned}\frac{}{\bar{n} \text{ val}} \text{ (VALNUM)} \quad & \frac{e_1 \mapsto e'_1 \quad op \in \{+, -, *, /\}}{e_1 \text{ op } e_2 \mapsto e'_1 \text{ op } e_2} \text{ (STEPSEARCHOPLEFT)} \\ \frac{e_2 \mapsto e'_2 \quad op \in \{+, -, *, /\}}{\bar{n}_1 \text{ op } e_2 \mapsto \bar{n}_1 \text{ op } e'_2} \text{ (STEPSEARCHOPRIGHT)} \quad & \frac{}{\bar{n}_1 + \bar{n}_2 \mapsto \overline{n_1 + n_2}} \text{ (STEPADD)} \\ \frac{}{\bar{n}_1 - \bar{n}_2 \mapsto \overline{n_1 - n_2}} \text{ (STEPSUB)} \quad & \frac{}{\bar{n}_1 * \bar{n}_2 \mapsto \overline{n_1 n_2}} \text{ (STEPMUL)} \quad & \frac{n_2 \neq 0}{\bar{n}_1 / \bar{n}_2 \mapsto \overline{n_1 / n_2}} \text{ (STEPDIV)}\end{aligned}$$

This is basically our expression language **E** from class, but with only integers and with subtraction, multiplication and (integer) division added. To save space and time, we have only two search rules: they apply for all 4 arithmetic operations in place of *op*. Note that rule STEPDIV, as we'd hope, allows division only if $\bar{n}_2$ is nonzero. That means that a division by zero, e.g., $\bar{1}/\bar{0}$, is a “stuck” expression: it's not a value and it can't step. So in order to make Progress and Preservation hold, we need a type system that rules out division by zero. The **EZ** language has only integers, so we don't need types to distinguish integers from strings, but we still have two types: **nz** (“not zero”) and **mz** (“maybe zero”), which we'll use to check whether an expression *might* evaluate to zero: the idea will be that if $e : \text{nz}$, then e definitely does not evaluate to zero (that is, if $e \mapsto^* \bar{n}$, then $n \neq 0$). We'll only allow division by expressions that definitely won't evaluate to zero.

Task 7 (6 points).

Fill in the blanks in the typing rules for **EZ**. The first two are done for you: any integer can have type **mz**, but it can only have **nz** if it's not 0. There are two rules for multiplication, depending on whether or not both operands are nonzero (think about why).

Hint: Think about, for each operation (addition, subtraction, multiplication, division), whether performing the operation on two integers can result in zero, depending on whether the two operands can be zero. *Integers can be negative.*

$$\begin{aligned}\frac{}{\bar{n} : \text{mz}} \text{ (TYPENUM)} \quad & \frac{n \neq 0}{\bar{n} : \text{nz}} \text{ (TYPENUMNZ)} \quad & \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 + e_2 : \text{?}} \text{ (TYPEADD)} \\ \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 - e_2 : \text{?}} \text{ (TYPESUB)} \quad & \frac{e_1 : \text{nz} \quad e_2 : \text{nz}}{e_1 * e_2 : \text{?}} \text{ (TYPEMULNZ)} \\ \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 * e_2 : \text{?}} \text{ (TYPEMULMZ)} \quad & \frac{e_1 : \text{mz} \quad e_2 : \text{?}}{e_1 / e_2 : \text{?}} \text{ (TYPEDIV)}\end{aligned}$$

You may find it concerning that, for example, there's no rule for $e_1 + e_2$ when e_1 has type **nz** and e_2 has type **mz**. This actually isn't a problem, because²:

Theorem 1. *For any expression e , if $e : \text{nz}$ then $e : \text{mz}$.*

The proof of this theorem isn't too difficult, but I won't make you do it.

The Canonical Forms lemma is similar to before, but now includes the fact that values of type **nz** are not zero.

²This is, in fact, a form of *subtyping*, which you may be familiar with from languages like Java, where whenever something has one type, it also has a more general type. We'll study this in more detail later in the course.

Lemma 1 (Canonical Forms). *If $e : \tau$ and e val, then:*

1. *If $\tau = \text{nz}$, then $e = \bar{n}$ for some $n \neq 0$.*
2. *If $\tau = \text{mz}$, then $e = \bar{n}$ for some n .*

As is typical, we won't prove Canonical Forms and won't feel too bad about it.
Here are the statements of Progress and Preservation for EZ.

Theorem 2 (Progress). *If $e : \tau$ then e val or there exists e' such that $e \mapsto e'$.*

Theorem 3 (Preservation). *If $e : \tau$ and $e \mapsto e'$ then $e' : \tau$.*

Task 8 (14 points).

Prove the Progress and Preservation theorems for EZ.

Hints/Comments – you'll want to read these.

- For Progress, you only need to prove the cases for `TYPE_NUM`, `TYPE_NUMNZ`, and `TYPE_DIV`^a.
- For Preservation, you only need to prove the cases for `STEP_ADD`, `STEP_SUB` and `STEP_DIV`^b.
- You can use without proof any (true!) facts of basic arithmetic.

^aFor a formal proof, the other cases for Progress need to be proven too, this is just me being nice and not making you do them :)

^bSame.

Bonus Task 1 (3 points).

Prove the `STEP_MUL` case for Preservation. **Hint:** if you do inversion on the typing rules, there will actually be *two* rules you need to consider in your inversion and thus, two cases.

Task 9 (4 points).

Give a syntactically valid expression in EZ that *does not* perform a division by zero, but is still *not well-typed* by the above typing rules. Informally suggest a change, addition, refinement, etc., to the type system that could make your expression well-typed (but would still rule out expressions that divide by zero). You don't need to actually implement your suggestion, or even argue that it's possible or practical, and it's fine if your solution still rules out some "OK" programs (programs that don't divide by zero). This is just to get you to think about how to get what you want with type systems.

4 One more wrap-up question

Task 10 (0 points).

How long (approximately) did you spend on this homework, in total minutes/hours of actual working time? Your honest feedback will help me with future homeworks.